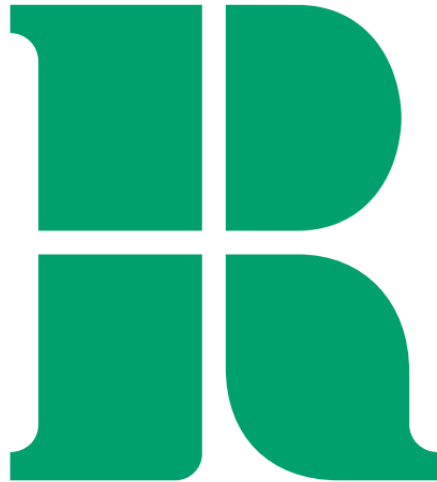




University of Roehampton London

School of Science, Technology
and Engineering

Course Name : MSc. Data Science

Module Title : Machine Learning

Module Code : CMP-L015-0

Module Tutor : Dr. Kuo-Ming Chao

Student ID : A00057300

Submission Date : 21.11.2025

Video Demonstration: [Machine learning cw2](#)

Contents

Introduction	3
Data Preparation Rationale	3
Model Architecture Justification	4
Critical Evaluation of results and Visualization	5
Conclusion	7
AI declaration	7
References	8

Figures

Figure 1: Overall Machine Learning Pipeline for the Flower Classification Task	3
Figure 2: creating batch size and normalizing using ImageDataGenerator	4
Figure 3: creating each of the split's separate folder directory to train CNN.....	4
Figure 4: Our model structure summary	4
Figure 5: Loss curve Train vs Validation over 10 Epochs	5
Figure 6: accuracy curve Train vs Validation sets over 10 Epochs	5
Figure 7: classification report of some of the class labels	5
Figure 8: scatterplot of the test split dataset	6
Figure 9: confusion matrix of the test dataset.....	6

Introduction

This study examines how a MobileNetV2-based transfer learning model performs on the Oxford-102 Flower dataset. A custom 80/10/10 split was used so the image-label mapping remained completely consistent, especially given the dataset's imbalance. Since many flower species in the dataset differ only in fine-grained details such as texture or lighting, the goal was to see how well a frozen-backbone network can cope with these subtle distinctions and what its learning behavior reveals about its representational limits.

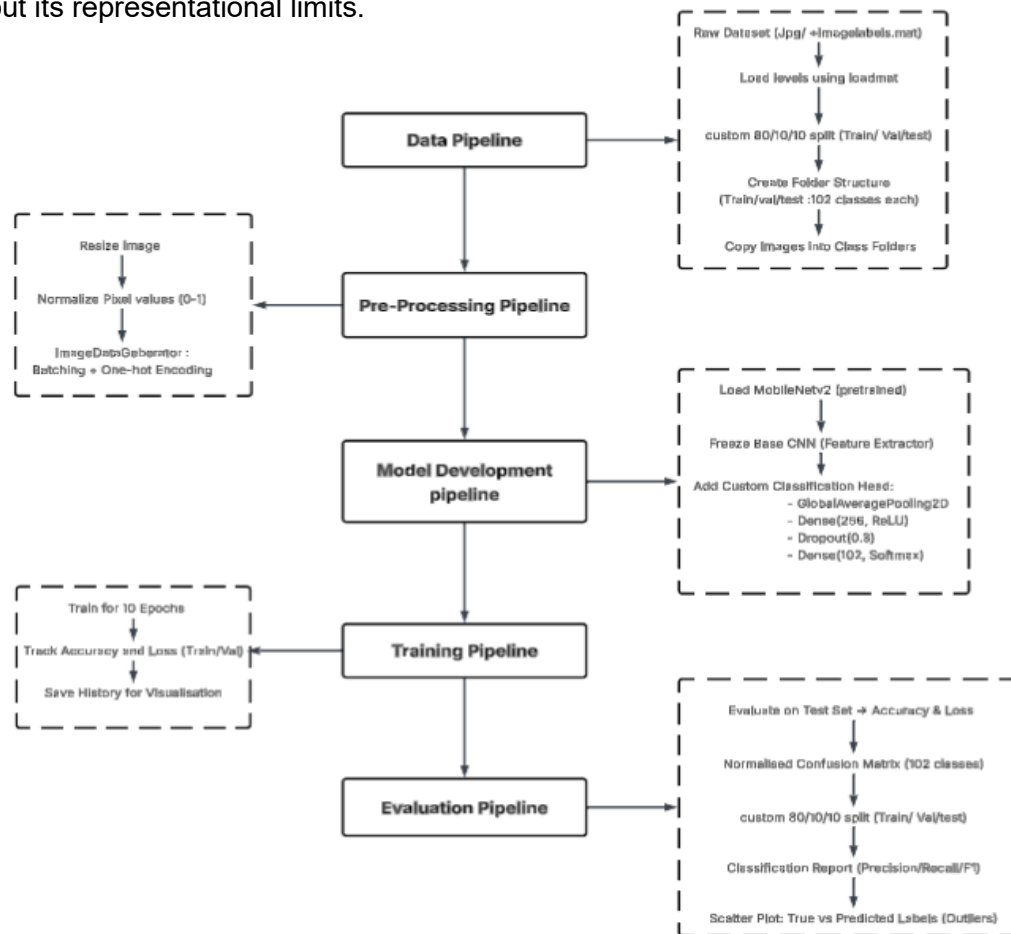


Figure 1: Overall Machine Learning Pipeline for the Flower Classification Task

Data Preparation Rationale

The dataset stores images in a single directory with labels encoded in external MATLAB arrays which were correctly aligned by converting them from 1-based to 0-based indexing. The images were reorganized into class-wise folders to provide the structural consistency required by KERAS

```
#creating a folder structure for the splitted dataset

base_dir = "/content/flowers_clean" #main directory to keep our cleaned dataset
splits=['train', 'val', 'test'] #dataset partitions

for s in splits:
    for cls in range(102):
        path=f"{base_dir}/{s}/{cls}"
        os.makedirs(path, exist_ok=True)
print('folder structure created successfully!')
```

Figure 3: creating each of the split's separate folder directory to train CNN

```
#preparing the raw images for training.

img_size = (224, 224) # MobileNetV2 required input
batch_size = 32 #number of image processed in one batch

#ImageDataGenerator handles preprocessing automatically
#rescaled the pixel values from 0-255 to 0-1
datagen = ImageDataGenerator(rescale=1./255)
```

Figure 2: creating batch size and normalizing using ImageDataGenerator

and to ensure transparent mapping between filenames and categories. Every image was resized to 224×224, which fits the size MobileNetV2 expects and prevents distortions that might confuse the model. Normalizing pixels to [0,1] helped keep gradients stable during training. No augmentation was added; this was intentional so any limitations in the model’s performance could be traced back to the frozen backbone rather than artificial variety introduced by augmentation.

Model Architecture Justification

MobileNetV2 was chosen because its inverted residual blocks and depth wise-separable convolutions emphasize mid-level texture representation, a property essential for distinguishing flower species whose differences lie in subtle local patterns rather than global form. According to the model summary, it contains 2.61M parameters, but only 354k were trainable because the ImageNet backbone was frozen. This constraint reduced the risk of overfitting, especially for classes with very limited examples

Domain specific learning was therefore concentrated in the classifier head (GlobalAveragePooling, a 256-unit dense layer, dropout and SoftMax) were creating a controlled adaptation space that stabilizes training and enables clearer interpretation of the backbone’s representational limits.

Model: "sequential"

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense (Dense)	(None, 256)	327,936
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 102)	26,214

Total params: 2,612,134 (9.96 MB)
Trainable params: 354,150 (1.35 MB)
Non-trainable params: 2,257,984 (8.61 MB)

Figure 4: Our model structure summary

Critical Evaluation of results and Visualization

The model's early behavior reflects strong transferability. Validation accuracy rises to 0.86 within 3 epochs, showing that the backbone recognizes dominant color-texture combinations across species. Learning then saturates.

Validation accuracy stayed between 0.879–0.885, while training accuracy rose to 0.9738, revealing a noticeable gap. The fact that validation loss remained stuck around 0.4229 also shows that the network was no longer improving its representations.

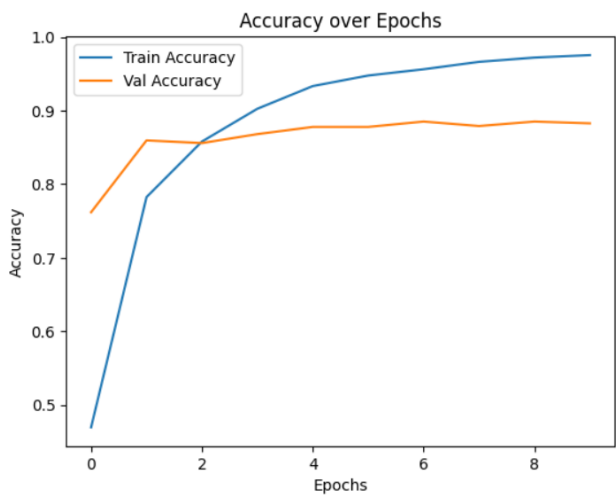


Figure 6: accuracy curve Train vs Validation sets over 10 Epochs

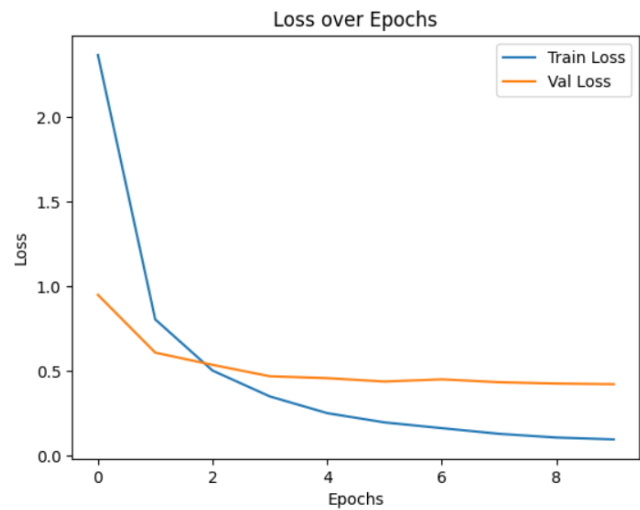


Figure 5: Loss curve Train vs Validation over 10 Epochs

The classification report highlights how the frozen backbone interacts with class-support variation. Several classes with only 2–4 test samples show unstable recall, but poor performance is not limited to low-support categories. For example, Class 97, which has seven test samples, still falls to a recall of 0.57, indicating that the backbone struggles with fine-texture cues rather than just sparse data.

97	0.80	0.57	0.67	7
98	0.89	0.89	0.89	9
99	0.88	0.88	0.88	8
100	1.00	1.00	1.00	8
101	0.80	1.00	0.89	4
accuracy			0.91	819
macro avg	0.91	0.90	0.90	819
weighted avg	0.92	0.91	0.91	819

Figure 7: classification report of some of the class labels

A few outliers appear in the scatter plot, which is expected due to the fine-grained similarity between classes, but most points align closely with the diagonal.

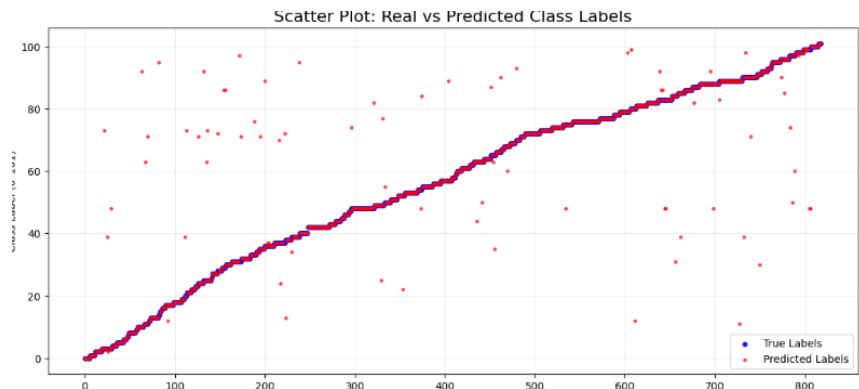


Figure 8: scatterplot of the test split dataset

The confusion matrix shows small but consistent misclassifications among yellow or low-contrast flowers, hinting that the model relies heavily on overall color similarity when finer texture cues are too subtle for the frozen backbone to capture.

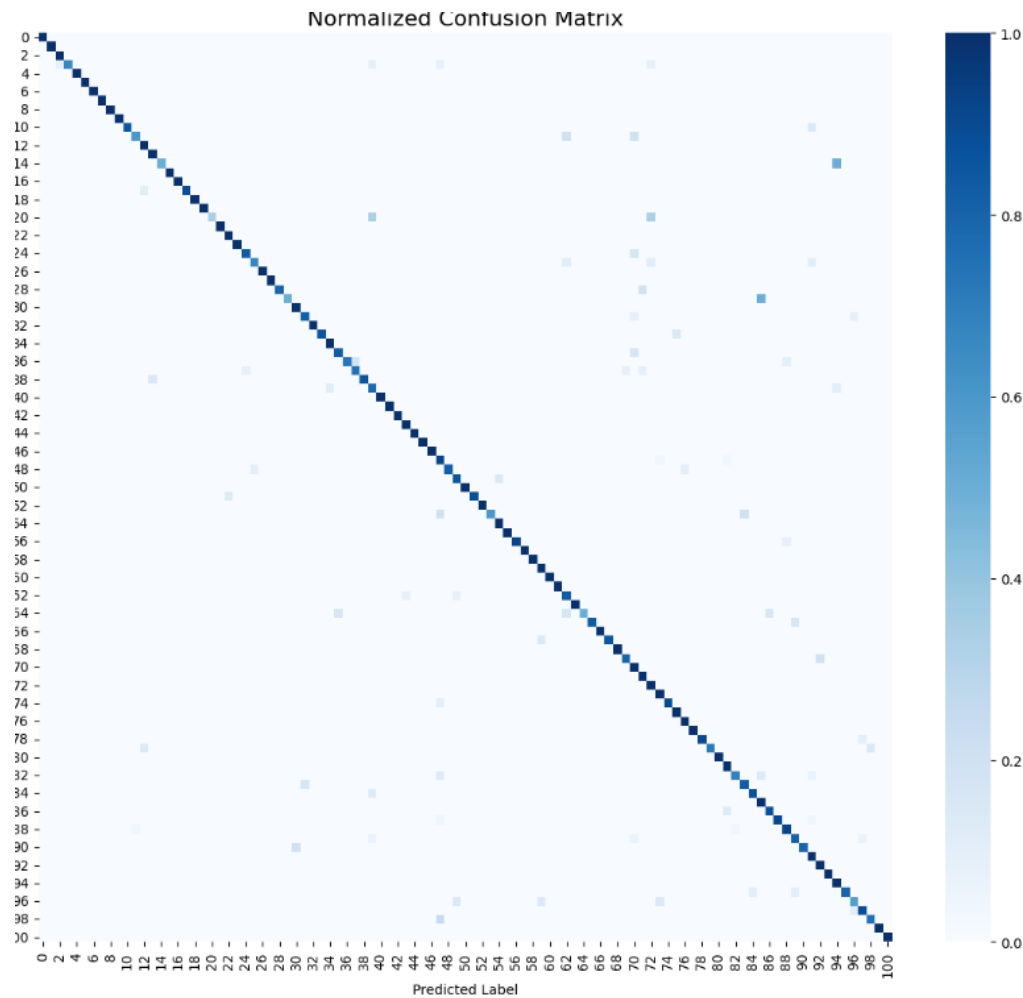


Figure 9: confusion matrix of the test dataset

Conclusion

Overall, the frozen MobileNetV2 backbone limited the model to coarse features, causing a performance ceiling and unstable behavior on classes with very few images. Low-light and low-contrast cases further exposed this weakness.

Future work should selectively unfreeze deeper layers, apply stronger color and contrast augmentation, and use class-balanced or focal loss to improve minority-class performance.

Although, the study shows that while transfer learning provides strong baselines, fine-grained classification requires targeted refinement beyond a fixed backbone.

AI declaration

I confirm that all modelling, coding, experimentation, and analysis presented in this coursework are my own work. Generative AI tools (such as ChatGPT, Grammarly) were used only to support language refinement, clarify code concepts, and improve the structure of the written report. The design decisions, interpretations, and conclusions are entirely my own.

References

1. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen,
“MobileNetV2: Inverted Residuals and Linear Bottlenecks,”
Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., pp. 4510–4520, 2018.
Available: <https://arxiv.org/abs/1801.04381>
Accessed: Nov. 15, 2024.
2. J. Ge, S. Huang, D. Li and X. Yan,
“Fine-Grained Image Classification via Deep Learning: A Survey,”
IEEE Access, vol. 7, pp. 123403–123415, 2019.
Available: <https://ieeexplore.ieee.org/document/8814701>
Accessed: Nov. 17, 2024.
3. P. Buda, A. Maki and M. Mazurowski,
“A Systematic Study of the Class Imbalance Problem in Convolutional Neural Networks,”
Neural Networks, vol. 106, pp. 249–259, 2018.
Available: <https://arxiv.org/abs/1710.05381>
Accessed: Nov. 14, 2024.
4. Y. Cui, M. Jia, T.-Y. Lin, Y. Song and S. Belongie,
“Class-Balanced Loss Based on Effective Number of Samples,”
Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., 2019.
Available: <https://arxiv.org/abs/1901.05555>
Accessed: Nov. 16, 2024.
5. M. A. Nilsback and A. Zisserman,
“A Visual Vocabulary for Flower Classification,”
Proc. IEEE Conf. Comput. Vis. Pattern Recognit., 2006.
Available: <https://www.robots.ox.ac.uk/~vgg/data/flowers/102/>
Accessed: Nov. 12, 2024.

